

FUTURE INTERNS

Cyber Security Internship | Task 1 | FUTURE_CS_01

VULNERABILITY ASSESSMENT REPORT

for a Live Website

CONFIDENTIAL — FOR EDUCATIONAL PURPOSES ONLY

Target Application:

<http://zero.webappsecurity.com>

Assessment Type: Passive Vulnerability Assessment (Grey-Box)

Prepared By: [Your Name]

Intern – Cyber Security | Future Interns

GitHub Repository: FUTURE_CS_01

Date: 23 May 2026

1. Non-Disclosure Statement

All information obtained during this security assessment including system configurations, identified vulnerabilities, and supporting evidence is deemed privileged and confidential. This report is produced solely for educational purposes as part of the Future Interns Cyber Security Internship program (Task 1 - FUTURE_CS_01). The contents will not be disclosed to any third party. The intern is committed to maintaining the highest ethical standards throughout their security practice.

2. Legal Notice

This assessment was conducted exclusively on <http://zero.webappsecurity.com> - a publicly accessible, intentionally vulnerable demo banking application maintained by HP Web Inspect for security training purposes. All scanning was performed passively using read-only tools. No vulnerabilities were exploited, no data was modified, and no aggressive or intrusive techniques were used. This report does not constitute a guarantee of the assessed application's security posture.

3. Executive Summary

In May 2026, a passive grey-box vulnerability assessment was conducted on <http://zero.webappsecurity.com> as Task 1 of the Future Interns Cyber Security Internship. The assessment used three tools: Nmap (network port and service scanning), OWASP ZAP (passive web scanning), and Browser DevTools (manual header and cookie inspection). A total of 8 unique vulnerability findings were identified across four risk categories.

Risk Level	Findings Count
High	0
Medium	4
Low	3
Informational	1
Total	8

The four medium-risk findings represent the most urgent issues: the absence of a Content-Security-Policy header leaves the application open to script injection attacks; Cross-Domain Misconfiguration (CORS set to wildcard) allows any external website to read responses from this application; the missing Anti-Clickjacking header enables UI deception attacks; and the use of an outdated jQuery library (v1.8.2) exposes users to known exploitable vulnerabilities. Three low-risk findings relate to server version information being disclosed. No high-risk vulnerabilities were found during passive scanning.

4. Introduction

The target, <http://zero.webappsecurity.com> (Zero Bank), is an intentionally vulnerable demo banking web application developed by HP WebInspect for security training. It simulates a realistic online banking environment with user login, account management, funds transfer, and bill payment functionality all of which present a meaningful attack surface for passive security assessment.

A passive grey-box assessment was conducted using three tools: Nmap for network-level reconnaissance (open ports and service fingerprinting), OWASP ZAP in passive scan mode (automated detection of HTTP-level security weaknesses), and Browser DevTools for manual inspection of HTTP response headers, cookies, and the JavaScript console. At no point were payloads injected, sessions hijacked, or server-side actions triggered maliciously.

This report presents all findings with their exact CWE/WASC identifiers as reported by ZAP, plain-language business impact explanations, DREAD risk scores, and practical remediation guidance.

5. Tools & Methodology

Tool	Version / Type	Purpose
Nmap	7.99 (Windows)	Port scanning, service & version detection
OWASP ZAP	Passive Scan Mode	Automated passive web vulnerability detection
Browser DevTools	Chrome F12	HTTP headers, cookies, console inspection

Nmap Commands Run

- Basic scan: `nmap zero.webappsecurity.com`
- Version scan: `nmap -sV zero.webappsecurity.com`

Nmap Results Summary

Target IP: 54.82.22.214 | rDNS: ec2-54-82-22-214.compute-1.amazonaws.com (AWS hosted)

Port	State	Service	Version Detected
80/tcp	Open	http	Apache Tomcat/Coyote JSP engine 1.1
443/tcp	Open	ssl/https?	Apache Tomcat/Coyote JSP engine 1.1
8080/tcp	Open	http-proxy	Apache Tomcat/Coyote JSP engine 1.1

6. Scope

6A. In-Scope

- Target URL: <http://zero.webappsecurity.com>
- Network reconnaissance: Nmap port scan and -sV version detection
- Passive web scanning: OWASP ZAP (8 alerts detected across all pages)
- Manual inspection: Browser DevTools — HTTP headers, cookies, JS console

6B. Out-of-Scope

- Active exploitation of any identified vulnerability
- Aggressive Nmap scans (-A, --script, -O flags)
- Any form of authentication bypass or session manipulation
- Denial of Service, fuzzing, or stress testing
- Any real production systems or third-party infrastructure

7. Risk Classification (DREAD Framework)

Each finding is scored using the DREAD model (Damage, Reproducibility, Exploitability, Affected Users, Discoverability) on a scale of 0 to 15.

	Factor	High (3)	Medium (2)	Low (1)
D	Damage Potential	Full compromise	Sensitive data leak	Trivial info only
R	Reproducibility	Always reproducible	Requires timing	Hard to reproduce
E	Exploitability	Novice attacker	Skilled attacker	Expert required
A	Affected Users	All users	Some users	Very few users
D	Discoverability	Publicly known	Hidden feature	Very obscure

Rating	Score Range
High	12 - 15
Medium	8 - 11
Low	5 - 7
Informational	0

8. Vulnerabilities Overview

#	Vulnerability	Risk	CWE	DREAD
1	Content Security Policy (CSP) Header Not Set	Medium	CWE-693	10/15
2	Cross-Domain Misconfiguration (CORS Wildcard)	Medium	CWE-264	9/15
3	Missing Anti-Clickjacking Header (X-Frame-Options)	Medium	CWE-1021	9/15
4	Vulnerable JS Library (jQuery 1.8.2)	Medium	CWE-1395	8/15
5	In Page Banner Information Leak (Tomcat/7.0.70)	Low	CWE-497	6/15
6	Server Version Disclosure via HTTP Response Header	Low	CWE-497	6/15
7	X-Content-Type-Options Header Missing	Low	CWE-693	5/15
8	Modern Web Application (Informational)	Informational	—	0

9. Detailed Vulnerability Findings

9.1 Content Security Policy (CSP) Header Not Set

URL	http://zero.webappsecurity.com/robots.txt (and all pages)
Risk	Medium (DREAD Score: 9/15)
Confidence	High
CWE ID	CWE-693
WASC ID	WASC-15
ZAP Source	Passive (10038 – Content Security Policy Header Not Set)
Evidence	No CSP header present in HTTP response
Description	Content Security Policy is a browser-enforced mechanism that restricts which scripts, styles, and resources can load on a page. Without it, if any Cross-Site Scripting (XSS) vulnerability exists or is introduced, the browser has no policy to block malicious scripts from executing. Attackers can steal session tokens, redirect users, or deface the page. ZAP flagged this with High confidence across all scanned URLs.
Attack Technique	No active exploitation is required. If any XSS entry point is found elsewhere in the application, the absence of CSP means injected scripts execute freely with no browser-level defence.
Countermeasures	Add the Content-Security-Policy HTTP response header in the web server (Tomcat) configuration. Start with: Content-Security-Policy: default-src 'self'; script-src 'self'. Use report-only mode first to test without breaking functionality: Content-Security-Policy-Report-Only.
Reference	OWASP A05:2021 – Security Misconfiguration CWE-693 WASC-15 ZAP Alert Ref: 10038-1
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.2 Cross-Domain Misconfiguration (CORS Wildcard)

URL	http://zero.webappsecurity.com/robots.txt
Risk	Medium (DREAD Score: 9/15)
Confidence	Medium
CWE ID	CWE-264
WASC ID	WASC-14
ZAP Source	Passive (10098 – Cross-Domain Misconfiguration)
Evidence	Access-Control-Allow-Origin: *

Description	The server responds with the CORS header Access-Control-Allow-Origin: * — a wildcard that allows any website on the internet to make cross-origin requests and read the responses from this application. For a banking application, this means a malicious website visited by a logged-in user could silently read account data or perform actions on their behalf. This is a serious misconfiguration for any application handling sensitive information.
Attack Technique	A malicious third-party site can use JavaScript's fetch() or XMLHttpRequest to call zero.webappsecurity.com endpoints from the victim's browser and read the responses, since the wildcard CORS header permits it.
Countermeasures	Replace the wildcard CORS header with an explicit allowlist of trusted domains. Change Access-Control-Allow-Origin: * to Access-Control-Allow-Origin: https://trusted-domain.com. Never use a wildcard on endpoints that handle authenticated or sensitive data.
Reference	OWASP A05:2021 – Security Misconfiguration CWE-264 WASC-14 ZAP Alert Ref: 10098
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.3 Missing Anti-Clickjacking Header (X-Frame-Options)

URL	http://zero.webappsecurity.com (all pages — 4 instances detected)
Risk	Medium (DREAD Score: 9/15)
Confidence	Medium
CWE ID	CWE-1021
WASC ID	WASC-15
ZAP Source	Passive (10020 – Anti-clickjacking Header)
Evidence	Parameter: x-frame-options (absent from all responses)
Description	The application does not return an X-Frame-Options header or a Content-Security-Policy frame-ancestors directive. Without this, any website can embed this banking application inside a hidden iframe. Attackers use this Clickjacking technique to overlay an invisible banking page over a decoy page, tricking the victim into clicking buttons they cannot see — for example, confirming a funds transfer without realising it.
Attack Technique	A malicious page hosts an invisible iframe pointing to the bank's funds transfer page. The victim, already logged in, clicks what they think is a game or quiz button — but is actually clicking the hidden 'Confirm Transfer' button on the banking site beneath it.
Countermeasures	Add the response header X-Frame-Options: DENY to prevent the page from being framed by any site. Alternatively, use Content-Security-Policy: frame-ancestors 'none' in modern browsers. This is a single-line change in the Tomcat server configuration or application web.xml.
Reference	OWASP A05:2021 – Security Misconfiguration CWE-1021 WASC-15 ZAP Alert Ref: 10020-1
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.4 Vulnerable JavaScript Library (jQuery 1.8.2)

URL	http://zero.webappsecurity.com/resources/js/jquery-1.8.2.min.js
Risk	Medium (DREAD Score: 9/15)
Confidence	Medium
CWE ID	CWE-1395
WASC ID	N/A
ZAP Source	Passive (10003 – Vulnerable JS Library, Powered by Retire.js)
Evidence	jquery-1.8.2.min.js
Description	The application loads jQuery version 1.8.2 — a library released in 2012 that has multiple known security vulnerabilities including Cross-Site Scripting (XSS)

	via jQuery.html() and prototype pollution issues. Retire.js (integrated into OWASP ZAP) flagged this library as outdated and vulnerable. Using a library over a decade old with published CVEs significantly increases the application's attack surface.
Attack Technique	Attackers who find an injection point can leverage known jQuery 1.8.2 CVEs to execute scripts. Published proof-of-concept exploits are freely available online, reducing the skill barrier for attackers.
Countermeasures	Update jQuery to the latest stable version (3.7.x or higher). Audit all other JavaScript dependencies using a tool such as npm audit or Retire.js. Implement a process to regularly review and update third-party libraries. Consider using Subresource Integrity (SRI) attributes on script tags.
Reference	OWASP A06:2021 – Vulnerable and Outdated Components CWE-1395 ZAP Alert Ref: 10003
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.5 In-Page Banner Information Leak (Tomcat/7.0.70)

URL	http://zero.webappsecurity.com/robots.txt (and page content)
Risk	Low (DREAD Score: 6/15)
Confidence	High
CWE ID	CWE-497
WASC ID	WASC-13
ZAP Source	Passive (10009 – In Page Banner Information Leak)
Evidence	Tomcat/7.0.70
Description	The server discloses the exact version of Apache Tomcat (7.0.70) within page response content or error banners. Apache Tomcat 7.0.70 was released in 2016 and reached end-of-life. There are multiple publicly known CVEs for this version including Remote Code Execution and information disclosure vulnerabilities. This version disclosure helps attackers immediately identify which exploits to use without guessing.
Attack Technique	An attacker reads the version banner and searches CVE databases (cve.mitre.org) for Tomcat 7.0.70 exploits. Multiple CVEs exist for this specific version with public proof-of-concept code.
Countermeasures	Upgrade Apache Tomcat to the latest stable supported version (10.x or higher). Suppress version banners by configuring server.xml: set the server attribute in the Connector element to a blank or generic value. Remove default error pages that display version information.
Reference	CWE-497 WASC-13 OWASP A06:2021 – Vulnerable and Outdated Components ZAP Alert Ref: 10009
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.6 Server Version Disclosure via HTTP Response Header

URL	http://zero.webappsecurity.com/sitemap.xml (and all responses)
Risk	Low (DREAD Score: 6/15)
Confidence	High
CWE ID	CWE-497
WASC ID	WASC-13
ZAP Source	Passive (10036 – HTTP Server Response Header)
Evidence	Apache-Coyote/1.1 (in Server HTTP response header)
Description	Every HTTP response from the server includes a Server header that explicitly discloses the web server software and version: Apache-Coyote/1.1. This is the HTTP connector component of Apache Tomcat. While this alone is not directly

	exploitable, it gives attackers precise information to search for known vulnerabilities specific to this component version, significantly reducing their reconnaissance effort.
Attack Technique	An attacker examines HTTP response headers using browser DevTools or a proxy tool. The Server header immediately reveals the exact server software, narrowing the attack to known CVEs for Apache-Coyote/1.1.
Countermeasures	Suppress the Server response header in Tomcat's configuration. In server.xml, add server="" to the Connector element to return a blank Server header. Alternatively configure the RemoteIpValve or a custom Filter to strip or replace the header value.
Reference	CWE-497 WASC-13 OWASP A05:2021 – Security Misconfiguration ZAP Alert Ref: 10036-2
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.7 X-Content-Type-Options Header Missing

URL	http://zero.webappsecurity.com (all pages)
Risk	Low (DREAD Score: 6/15)
Confidence	Medium
CWE ID	CWE-693
WASC ID	WASC-15
ZAP Source	Passive (10021 – X-Content-Type-Options Header Missing)
Evidence	Parameter: x-content-type-options (absent)
Description	The X-Content-Type-Options: nosniff header is absent from server responses. Without it, browsers may attempt to 'MIME-sniff' the content type of responses — overriding what the server declares. This allows attackers to deliver a file with a misleading extension (e.g., a .jpg containing JavaScript) that the browser then executes as a script. This vulnerability was flagged by ZAP as affecting the main application URL.
Attack Technique	An attacker uploads or links to a file with a safe extension (e.g., .txt or .jpg) that actually contains JavaScript. Without the nosniff header, certain browsers execute the content as a script if they detect script-like content during MIME sniffing.
Countermeasures	Add the response header X-Content-Type-Options: nosniff to all HTTP responses. This is a single-line addition in Tomcat's web.xml via the HttpHeaderSecurityFilter, or can be applied globally in a servlet filter. It has no impact on normal application functionality.
Reference	CWE-693 WASC-15 OWASP A05:2021 – Security Misconfiguration ZAP Alert Ref: 10021
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

9.8 Modern Web Application (Informational)

URL	http://zero.webappsecurity.com
Risk	Informational (DREAD Score: 0/15)
Confidence	Medium
CWE ID	—
WASC ID	—
ZAP Source	Passive (10109 – Modern Web Application)
Evidence	More Services
Description	ZAP detected that this is a modern web application using dynamic HTML elements and JavaScript-driven navigation (Bootstrap/jQuery UI patterns). This

	is an informational note — not a vulnerability. It indicates that the AJAX Spider may be more effective than the standard spider for complete coverage in future active scanning sessions. No security risk is associated with this finding.
Attack Technique	No attack technique applicable — this is an informational fingerprint of the application framework.
Countermeasures	No remediation required. If conducting future active scanning, use the AJAX Spider in OWASP ZAP to ensure full application coverage including dynamically loaded content.
Reference	ZAP Alert Ref: 10109 — Informational only
Screenshot	See screenshots/ folder in GitHub repository: FUTURE_CS_01

10. Conclusion & Remediation Roadmap

The passive vulnerability assessment of <http://zero.webappsecurity.com> identified 8 findings — 4 medium-risk, 3 low-risk, and 1 informational. All findings are directly based on evidence collected from real tool outputs: Nmap version scan results, OWASP ZAP passive alerts (with exact CWE/WASC IDs), and Browser DevTools observations. No active exploitation was performed at any stage.

The four medium-risk findings are the priority. The CORS wildcard misconfiguration is particularly concerning for a banking application as it allows any website to read authenticated responses. The missing CSP and Anti-Clickjacking headers are single-line server configuration fixes that would significantly improve the application's defence posture. Updating the outdated jQuery library is a straightforward dependency management task.

Prioritised Remediation Roadmap:

- IMMEDIATE — Fix CORS: Replace Access-Control-Allow-Origin: * with an explicit trusted-domain allowlist
- IMMEDIATE — Add CSP header: Content-Security-Policy: default-src 'self'; script-src 'self'
- IMMEDIATE — Add Anti-Clickjacking: X-Frame-Options: DENY on all responses
- SHORT-TERM — Update jQuery 1.8.2 to latest stable version (3.7.x+)
- SHORT-TERM — Add X-Content-Type-Options: nosniff to all responses
- MEDIUM-TERM — Upgrade Apache Tomcat from 7.0.70 (EOL) to latest supported version
- MEDIUM-TERM — Suppress Server and version banner headers in Tomcat configuration

Assessment conducted ethically within the authorized scope of the Future Interns Cyber Security Internship. Target is a legal, intentionally vulnerable demo application. All scan outputs and screenshots are documented in GitHub repository FUTURE_CS_01.